
Testes Unitários — Inventory Masters Kinect

SUMÁRIO

SUMÁRIO	1
1. Identificação	3
2. Testes Unitários — Inventory Masters Kinect	3
2.1 Justificativa da escolha da ViewModel	3
2.2 Estratégia de testes	4
2.2.1 Arrange	4
2.2.2 Act	4
2.2.3 Assert	4
2.3 Casos de teste	5
2.3.1 BaseViewModel	5
2.3.2 KinectLoginViewModel	5
2.3.3 MainViewModel	6
2.3.4 MainViewModel — Espaço	6
2.3.5 MainViewModel — Volume	7
2.3.6 MainViewModel — Histórico	8
2.3.7 MainViewModel — Interface e sessão	8
2.3.8 MainViewModel — Kinect	9
2.4 Técnicas utilizadas	9
2.4.1 Testes de cenários válidos	9
2.4.2 Testes de cenários inválidos	9
2.4.3 Análise de valores-limite	9
2.4.4 Testes de exceções	10
2.4.5 Testes parametrizados	10
2.4.6 Test doubles	10
2.4.7 Injeção de dependências	10
2.5 Resultados dos testes do Kinect	10
2.6 Problemas encontrados e correções	10
2.7 Situações não testadas	11
2.8 Conclusão dos testes do Kinect	11
3. Testes Unitários — Inventory Masters MVC	12

3.1 Componentes testados no MVC.....	12
3.1.1 MedicaoHub.....	12
3.1.2 NotificacaoHub	13
3.1.3 ViewModels do MVC.....	13
3.2 Justificativa dos testes do MVC	13
3.3 Estratégia de testes do MVC	14
3.4 Casos de teste do MedicaoHub.....	14
3.4.1 Importância dos testes do MedicaoHub.....	14
3.5 Casos de teste do NotificacaoHub	15
3.6 Casos de teste das ViewModels do MVC.....	15
3.6.1 LoginEmailViewModel	15
3.6.2 SolicitarTokenKinectRequest.....	15
3.6.3 ValidarTokenViewModel.....	15
3.6.4 ValidarTokenRequest	16
3.6.5 ValidacaoTokenResultadoViewModel	16
3.6.6 DashboardViewModel	16
3.6.7 ErrorViewModel	16
3.7 Técnicas utilizadas nos testes do MVC.....	16
3.8 Problema encontrado e correção	17
3.8.1 Comportamento esperado.....	17
3.8.2 Comportamento encontrado	17
3.8.3 Correção realizada.....	17
3.9 Resultados dos testes do MVC	17
3.10 Situações não testadas no MVC	18
4. Resultado Geral	18
5. Conclusão Geral.....	18



1. Identificação

Nome do aluno: Diulie Mileide Batista Coreia

Equipe: Inventory Masters

Data: 23/08/2026

Projeto: TCC Inventory Masters Kinect

Componente escolhido: ViewModel

Classes testadas: BaseViewModel, KinectLoginViewModel e MainViewModel, incluindo suas classes parciais de espaço, histórico, interface, Kinect e volume.

2. Testes Unitários — Inventory Masters Kinect

O componente escolhido para a realização dos testes foi a camada ViewModel da aplicação Inventory Masters Kinect.

A ViewModel é responsável pela comunicação entre as Views, desenvolvidas em XAML, e as regras e os serviços utilizados pela aplicação. Ela controla os dados exibidos na tela, os comandos executados pelos botões, as mensagens apresentadas ao usuário e os estados da interface.

Entre as responsabilidades testadas estão:

- Notificação de alterações de propriedades para a interface;
- Controle dos comandos da aplicação;
- Solicitação e validação de tokens;
- Criação da sessão do usuário;
- Bloqueio e desbloqueio da sessão;
- Validação dos dados de espaço;
- Cálculo e exibição do volume;
- Carregamento do histórico de medições;
- Tratamento da conexão com o Kinect;
- Tratamento de erros dos serviços.

A escolha da ViewModel foi feita porque ela possui comportamentos importantes para o funcionamento da aplicação. Um erro nessa camada pode gerar dados incorretos na tela, permitir entradas inválidas, impedir o login ou causar falhas durante uma medição.

2.1 Justificativa da escolha da ViewModel

A ViewModel foi escolhida porque concentra grande parte da lógica utilizada pela interface do sistema.

Antes da separação da lógica, alguns comportamentos estavam presentes nos arquivos XAML.cs. Com a aplicação do padrão MVVM, esses comportamentos foram transferidos para classes ViewModel, deixando as Views responsáveis principalmente pela apresentação.

A criação dos testes permite verificar essas regras sem a necessidade de abrir todas as telas manualmente. Também possibilita avaliar situações de erro, valores-limite e falhas de serviços de maneira controlada.

2.2 Estratégia de testes

A suíte foi desenvolvida utilizando o framework xUnit e o padrão Arrange, Act e Assert.

2.2.1 Arrange

Nesta etapa são preparados:

- A ViewModel que será testada;
- Os dados de entrada;
- A sessão do usuário;
- Os serviços falsos utilizados pelo teste;
- O estado inicial esperado.

2.2.2 Act

Nesta etapa é executado o comportamento que está sendo testado, como:

- Alterar uma propriedade;
- Executar um comando;
- Solicitar um token;
- Validar um token;
- Bloquear ou desbloquear a sessão;
- Calcular o percentual de ocupação;
- Carregar o histórico.

2.2.3 Assert

Nesta etapa é verificado se o resultado corresponde ao comportamento esperado, como:

- Valor de uma propriedade;
- Mensagem exibida;
- Estado da sessão;
- Disparo do evento PropertyChanged;
- Habilitação de comandos;
- Quantidade de registros;
- Tratamento correto de exceções.

Foram criados serviços falsos, também chamados de fakes, para evitar que os testes dependessem do servidor MVC, envio de e-mail, banco de dados ou equipamento Kinect.

Dessa forma, os testes são unitários, repetíveis e podem ser executados sem depender de recursos externos.

2.3 Casos de teste

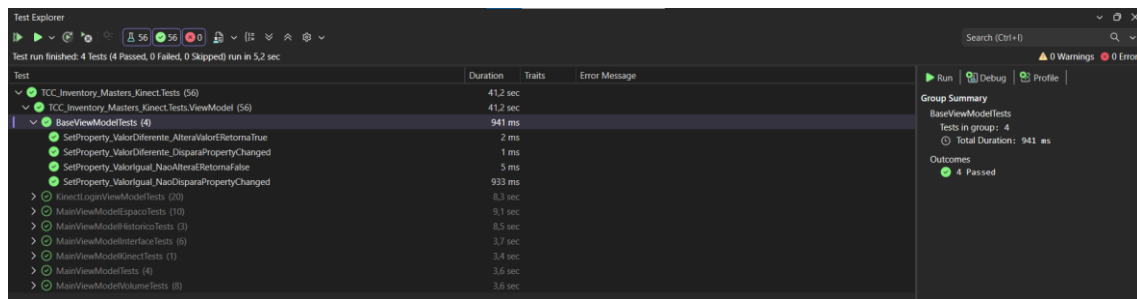
2.3.1 BaseViewModel

Os testes da classe BaseViewModel verificam o funcionamento da notificação de propriedades utilizada pelas demais ViewModels.

Foram testados os seguintes comportamentos:

- Alteração de uma propriedade;
- Disparo do evento PropertyChanged;
- Não disparar o evento quando o valor permanece igual;
- Retorno correto do método responsável por atualizar uma propriedade.

Esses testes são importantes porque a interface depende dessas notificações para atualizar os componentes visuais.

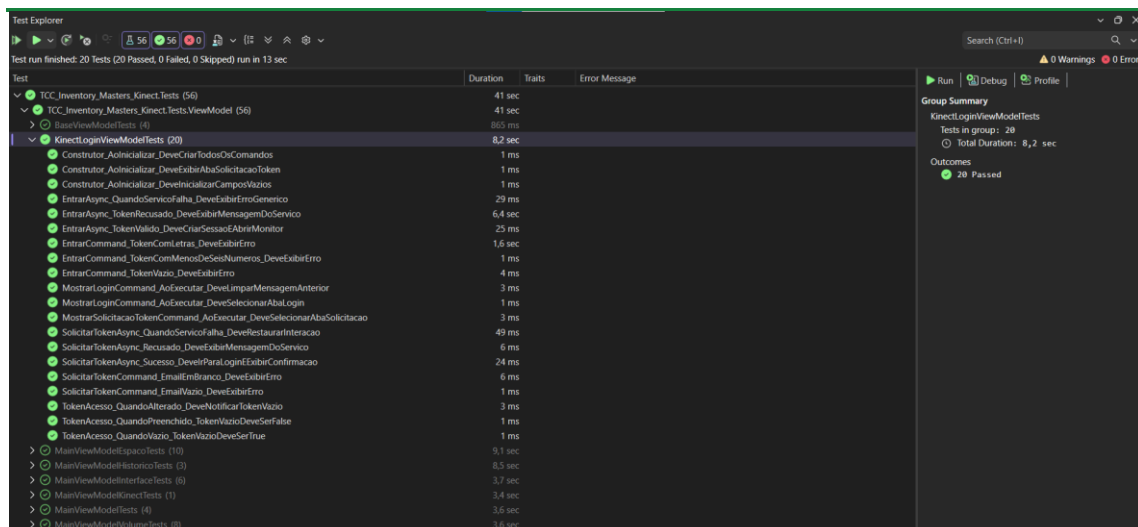


2.3.2 KinectLoginViewModel

Os testes de login verificam:

- Estado inicial da tela;
- Existência dos comandos;
- Alternância entre as etapas de solicitação e validação;
- E-mail vazio;
- Token vazio;
- Token com quantidade inválida de caracteres;
- Token contendo caracteres inválidos;
- Limpeza das mensagens anteriores;
- Token válido;
- Token recusado pelo serviço;
- Exceção durante a validação;
- Solicitação de token realizada com sucesso;
- Solicitação recusada pelo serviço;
- Exceção durante a solicitação;
- Criação correta da sessão;
- Abertura da tela principal após o login.

Esses testes são importantes porque impedem a validação de dados incompletos e verificam se falhas do serviço são informadas corretamente ao usuário.

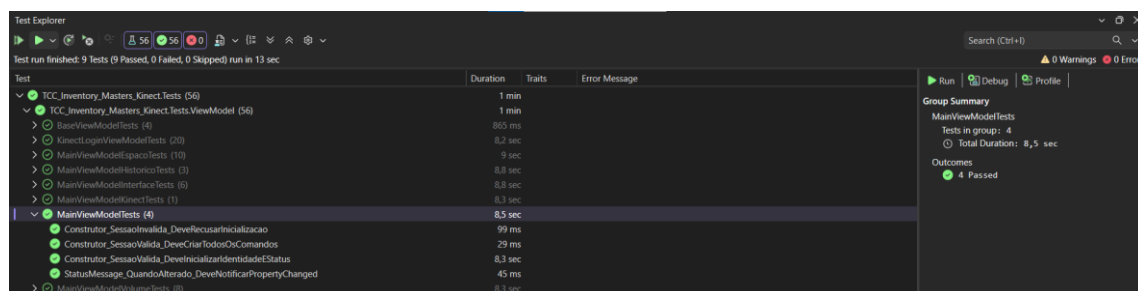


2.3.3 MainViewModel

Os testes principais da MainViewModel verificam:

- Rejeição de sessões inválidas;
- Inicialização com uma sessão válida;
- Valores iniciais das propriedades;
- Criação dos comandos da interface;
- Notificação de alteração das propriedades;
- Estado inicial do monitoramento.

Foram utilizadas diferentes combinações de dados de sessão, incluindo valores nulos, vazios e incompletos.



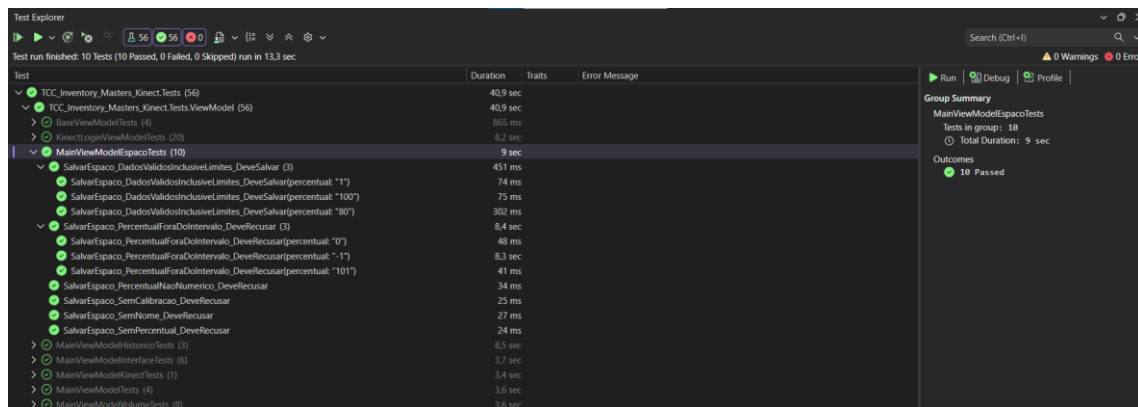
2.3.4 MainViewModel — Espaço

Os testes relacionados ao espaço verificam:

- Nome do espaço vazio;
- Percentual não informado;
- Percentual não numérico;
- Percentual igual a zero;
- Percentual negativo;
- Percentual maior que 100;

- Ausência de calibração;
- Valores válidos;
- Limite mínimo válido de 1%;
- Valor intermediário de 80%;
- Limite máximo válido de 100%.

Esses testes garantem que somente configurações válidas sejam utilizadas pela aplicação.

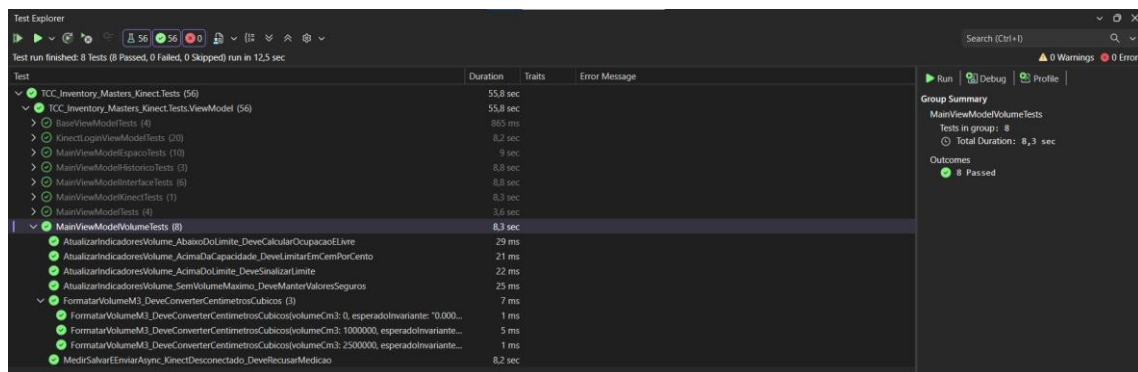


2.3.5 MainViewModel — Volume

Os testes de volume verificam:

- Conversão de centímetros cúbicos para metros cúbicos;
- Formatação do volume;
- Volume máximo não configurado;
- Volume abaixo do limite;
- Volume acima do limite;
- Limitação do percentual máximo em 100%;
- Tentativa de medição com o Kinect desconectado.

Foram testados valores como zero, 1.000.000 cm³ e 2.500.000 cm³.

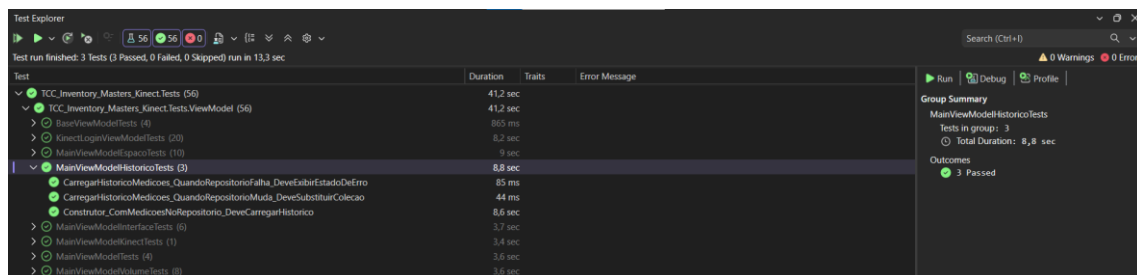


2.3.6 MainViewModel — Histórico

Os testes do histórico verificam:

- Carregamento inicial das medições;
- Atualização da coleção apresentada na tela;
- Substituição dos dados antigos pelos dados recebidos;
- Tratamento de exceção no repositório;
- Apresentação de mensagem de erro.

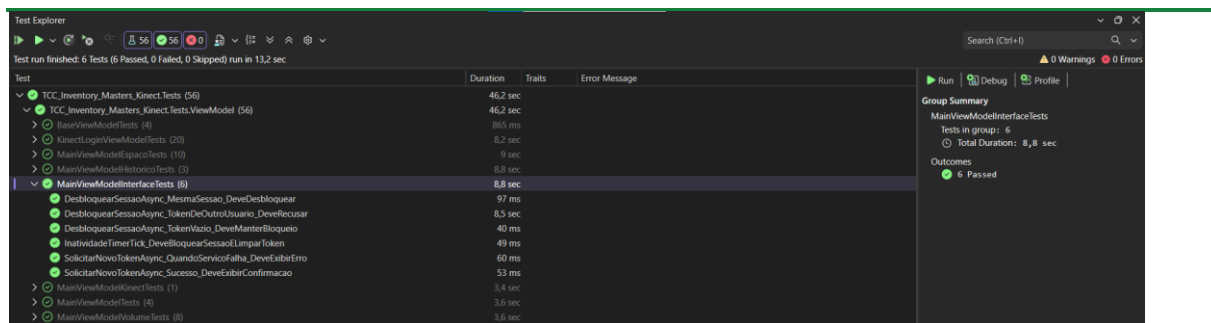
O repositório real foi substituído por um repositório falso para que os testes não dependessem do banco de dados.



2.3.7 MainViewModel — Interface e sessão

Os testes da interface verificam:

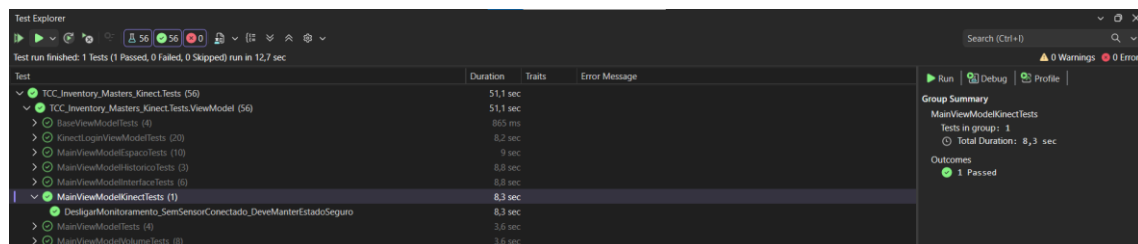
- Bloqueio da aplicação após inatividade;
- Tentativa de desbloqueio com token vazio;
- Recusa de token pertencente a outro usuário;
- Desbloqueio com token pertencente à mesma sessão;
- Comparação do identificador da sessão sem diferenciar letras maiúsculas e minúsculas;
- Solicitação de novo token;
- Mensagem de confirmação;
- Tratamento de falha durante a solicitação.



2.3.8 MainViewModel — Kinect

Os testes relacionados ao Kinect verificam se a aplicação consegue encerrar o monitoramento com segurança quando nenhum sensor está conectado.

As operações que dependem diretamente da leitura física do Kinect não foram executadas como testes unitários, pois necessitam de hardware e devem ser verificadas por testes de integração ou testes manuais.



2.4 Técnicas utilizadas

2.4.1 Testes de cenários válidos

Foram utilizados dados corretos para verificar o funcionamento esperado dos comandos, validações e cálculos.

2.4.2 Testes de cenários inválidos

Foram utilizados campos vazios, valores não numéricos, tokens inválidos e sessões incompletas.

2.4.3 Análise de valores-limite

Foram verificados limites como:

- Percentual de 0%;
- Percentual de 1%;
- Percentual de 100%;

- Percentual de 101%;
- Volume igual a zero;
- Percentual de ocupação limitado a 100%.

2.4.4 Testes de exceções

Os serviços falsos foram configurados para lançar exceções. Isso permitiu verificar se a ViewModel trata o erro e apresenta uma mensagem adequada.

2.4.5 Testes parametrizados

Foram utilizados Theory e InlineData para executar o mesmo comportamento com diferentes valores de entrada.

2.4.6 Test doubles

Foram criados serviços e repositórios falsos para representar:

- Serviço de autenticação;
- Repositório de medições;
- Resultado de solicitação de token;
- Resultado de validação de token.

2.4.7 Injeção de dependências

As dependências utilizadas pelas ViewModels podem ser fornecidas durante os testes. Isso evita chamadas reais para o servidor, e-mail, banco de dados e Kinect.

2.5 Resultados dos testes do Kinect

A suíte final apresentou o seguinte resultado:

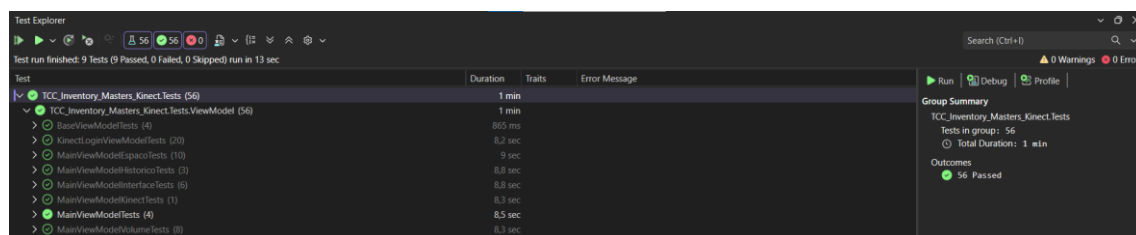
Total de testes: 61

Testes executados: 61

Testes aprovados: 61

Testes reprovados: 0

Resultado final: suíte aprovada com sucesso.



2.6 Problemas encontrados e correções

Durante a preparação da suíte foram encontrados problemas de configuração no projeto de testes.

O projeto estava utilizando uma configuração de framework incompatível com a aplicação WPF. A aplicação utiliza .NET Framework 4.8, enquanto o projeto de testes estava configurado de maneira diferente.

Também existia uma referência do Kinect utilizando um caminho específico de outra máquina. Isso poderia impedir a compilação em outros computadores.

Esses problemas foram corrigidos com:

- Configuração do projeto de testes para .NET Framework 4.8;
- Configuração da plataforma para x64;
- Correção da referência da biblioteca Microsoft.Kinect.dll;
- Inclusão correta do projeto de testes na solução.

Durante a primeira execução, um teste de formatação do volume apresentou falha. O teste esperava um separador decimal diferente daquele definido pelo comportamento atual da aplicação.

Após a análise, foi identificado que o código utiliza a representação `0.000 m³`. A expectativa do teste foi ajustada para refletir corretamente o comportamento especificado.

Depois da correção, todos os 61 testes foram executados com sucesso.

2.7 Situações não testadas

Não foram realizados testes unitários das seguintes situações:

- Captura física de dados pelo Kinect;
- Movimento real do usuário diante do sensor;
- Falha física do equipamento;
- Envio real de e-mail pelo Gmail;
- Comunicação real com o servidor MVC;
- Gravação real no Firebase;
- Aparência visual das telas XAML.

Essas situações não fazem parte dos testes unitários porque dependem de equipamentos, rede, serviços externos ou avaliação visual. Elas devem ser verificadas por testes de integração, testes de interface ou testes manuais.

2.8 Conclusão dos testes do Kinect

A implementação da suíte permitiu verificar os principais comportamentos das ViewModels da aplicação Inventory Masters Kinect.



Os testes validam cenários de sucesso, falha, valores inválidos, valores-limite e exceções. Também verificam comportamentos relacionados ao login, sessão, espaço, histórico, volume e conexão com o Kinect.

A utilização de serviços falsos e injeção de dependências permitiu executar os testes sem acessar o banco de dados, enviar e-mails ou conectar o equipamento Kinect.

O resultado final de 61 testes aprovados demonstra que os comportamentos avaliados estão funcionando conforme o esperado.

A atividade também demonstrou que os testes unitários não servem apenas para aumentar a quantidade de código do projeto. Eles ajudam a identificar problemas, documentar regras de negócio, facilitar futuras alterações e aumentar a confiabilidade da aplicação.

A suíte criada contribui para a qualidade do projeto porque permite verificar rapidamente se novas modificações causaram algum problema nos comportamentos que já estavam funcionando.

3. Testes Unitários — Inventory Masters MVC

Além das ViewModels do aplicativo Kinect, foram testados os Hubs e as ViewModels do projeto MVC Inventory Masters.

Os componentes adicionais testados foram:

- MedicaoHub;
- NotificacaoHub;
- BaseViewModel;
- DashboardViewModel;
- LoginEmailViewModel;
- SolicitarTokenKinectRequest;
- ValidarTokenViewModel;
- ValidarTokenRequest;
- ValidacaoTokenResultadoViewModel;
- ErrorViewModel.

3.1 Componentes testados no MVC

3.1.1 MedicaoHub

O MedicaoHub é responsável por receber as medições enviadas pelo Kinect por meio do SignalR.

Entre suas responsabilidades estão:

- Receber o volume em centímetros cúbicos;
- Converter o volume para metros cúbicos;
- Salvar a medição;

-
- Verificar se o volume atingiu o percentual de alerta;
 - Criar uma notificação automática;
 - Evitar notificações pendentes duplicadas;
 - Transmitir a nova medição para os clientes conectados;
 - Informar erros de processamento ao cliente que enviou a medição.

3.1.2 NotificacaoHub

O NotificacaoHub é responsável por enviar notificações em tempo real para todos os clientes conectados ao sistema.

Também possui métodos relacionados à conexão e desconexão dos clientes.

3.1.3 ViewModels do MVC

As ViewModels do MVC são utilizadas para transportar e validar os dados apresentados ou recebidos pelas Views e APIs.

Elas controlam informações como:

- E-mail utilizado para solicitar token;
- Token informado pelo usuário;
- Resultado da validação do token;
- Dados exibidos no Dashboard;
- Quantidade de medições, alertas, parceiros e usuários;
- Ordenação das notificações mais recentes;
- Identificação do usuário e da página;
- Identificação de erros da requisição.

3.2 Justificativa dos testes do MVC

Os Hubs foram escolhidos porque realizam operações importantes para a comunicação entre o Kinect e o sistema MVC.

Um erro no MedicaoHub poderia causar:

- Conversão incorreta do volume;
- Perda da medição;
- Criação duplicada de notificações;
- Ausência de alertas;
- Falha na atualização do Dashboard;
- Ausência de uma mensagem adequada quando ocorrer um erro.

As ViewModels do MVC foram testadas porque recebem dados fornecidos pelo usuário e organizam as informações exibidas nas Views.

Os testes ajudam a impedir que dados inválidos sejam processados e verificam se os indicadores apresentados no Dashboard correspondem aos dados armazenados.

3.3 Estratégia de testes do MVC

Os testes foram desenvolvidos com xUnit e organizados utilizando Arrange, Act e Assert.

Para testar os Hubs sem acessar o Firebase e sem iniciar uma conexão SignalR real, foram criadas implementações falsas das seguintes dependências:

- Repositório de medições;
- Repositório de parâmetros;
- Repositório de notificações;
- Clientes do SignalR;
- Contexto da conexão.

Também foram criadas interfaces para os repositórios utilizados pelo MedicaoHub. Isso permitiu substituir os repositórios reais por objetos controlados durante os testes.

Nenhum teste do MVC envia e-mail, acessa o Firebase ou estabelece uma conexão SignalR real.

3.4 Casos de teste do MedicaoHub

Os testes do MedicaoHub verificam os seguintes comportamentos:

- Conversão de zero centímetros cúbicos;
- Conversão de 1.000.000 cm³ para 1 m³;
- Conversão de 2.500.000 cm³ para 2,5 m³;
- Armazenamento da origem da medição como Kinect;
- Armazenamento do status inicial como Normal;
- Registro da data e hora da medição;
- Envio da medição para todos os clientes conectados;
- Volume abaixo do percentual de alerta;
- Volume exatamente no limite do alerta;
- Criação de uma notificação pendente;
- Prevenção de notificações duplicadas;
- Ausência dos parâmetros do sistema;
- Capacidade máxima igual a zero;
- Capacidade máxima negativa;
- Falha ao salvar uma medição;
- Envio de mensagem de erro somente para o cliente responsável;
- Falha durante a criação da notificação;
- Continuidade do envio da medição mesmo quando a notificação falha;
- Conexão de um cliente;
- Desconexão normal;
- Desconexão causada por uma exceção.

3.4.1 Importância dos testes do MedicaoHub

Esses testes garantem que o volume enviado pelo Kinect seja convertido corretamente e que os clientes recebam os dados esperados.



Os testes de alerta também garantem que uma notificação seja criada somente quando o percentual configurado for atingido e quando ainda não existir uma notificação pendente.

3.5 Casos de teste do NotificacaoHub

Os testes do NotificacaoHub verificam:

- Envio da notificação para todos os clientes;
- Nome correto do método SignalR utilizado pelo cliente;
- Conteúdo da mensagem enviada;
- Conexão de um cliente;
- Desconexão de um cliente.

Esses testes garantem que as mensagens em tempo real sejam encaminhadas pelo método correto.

3.6 Casos de teste das ViewModels do MVC

3.6.1 LoginEmailViewModel

Foram testados:

- E-mail vazio;
- E-mail em formato inválido;
- E-mail válido;
- Nome de exibição do campo.

3.6.2 SolicitarTokenKinectRequest

Foram testados:

- E-mail vazio;
- E-mail inválido;
- E-mail válido.

3.6.3 ValidarTokenViewModel

Foram testados:

- Token nulo;
- Token vazio;
- Token com menos de seis caracteres;
- Token com exatamente seis caracteres;
- Token com mais de seis caracteres;
- Token com doze caracteres;
- Token com treze caracteres.

3.6.4 ValidarTokenRequest

Foi verificado se o token recebido pela API é armazenado corretamente na propriedade correspondente.

3.6.5 ValidacaoTokenResultadoViewModel

Foi verificado o armazenamento dos seguintes resultados:

- Token válido;
- E-mail validado;
- Nome do usuário;
- Empresa;
- E-mail;
- Mensagem da validação.

3.6.6 DashboardViewModel

Foram testados:

- Inicialização das listas;
- Inicialização dos parâmetros;
- Mensagem de erro inicial;
- Total de medições;
- Total de alertas;
- Total de parceiros;
- Total de usuários;
- Comportamento quando as coleções são nulas;
- Retorno das cinco notificações mais recentes;
- Ordenação das notificações pela data;
- Nome do usuário;
- Título da página;
- Percentual de ocupação;
- Parâmetros do sistema;
- Mensagem de erro controlada.

3.6.7 ErrorViewModel

Foram testados:

- Identificador da requisição nulo;
- Identificador vazio;
- Identificador preenchido;
- Exibição condicional do identificador da requisição.

3.7 Técnicas utilizadas nos testes do MVC

Foram utilizadas as seguintes técnicas:

-
- Cenários válidos;
 - Cenários inválidos;
 - Análise de valores-limite;
 - Testes parametrizados com Theory e InlineData;
 - Testes de exceções;
 - Test doubles;
 - Repositórios falsos;
 - Contexto SignalR falso;
 - Injeção de dependências;
 - Verificação de coleções;
 - Verificação de mensagens transmitidas;
 - Verificação de propriedades;
 - Verificação de validações com Data Annotations.

3.8 Problema encontrado e correção

Durante os testes da `ValidarTokenViewModel`, foi encontrada uma inconsistência na validação do token.

A mensagem apresentada pela `ViewModel` informava que o token deveria possuir seis caracteres. O serviço responsável pela geração do token também cria um token numérico de exatamente seis dígitos.

Entretanto, a configuração da propriedade permitia tokens entre seis e doze caracteres.

3.8.1 Comportamento esperado

Aceitar somente tokens com exatamente seis caracteres.

3.8.2 Comportamento encontrado

Tokens entre seis e doze caracteres eram aceitos pela validação da `ViewModel`.

3.8.3 Correção realizada

A validação foi alterada para exigir o tamanho mínimo e máximo de seis caracteres.

Depois da correção, foram executados novamente os testes de limite:

- Cinco caracteres: inválido;
- Seis caracteres: válido;
- Sete caracteres: inválido;
- Doze caracteres: inválido;
- Treze caracteres: inválido.

3.9 Resultados dos testes do MVC

A execução final da suíte do MVC apresentou:

Total de testes: 40
Testes aprovados: 40
Testes reprovados: 0
Testes ignorados: 0
Resultado: suíte aprovada com sucesso.

Somando os testes do MVC com os testes das ViewModels do aplicativo Kinect, o projeto possui:

Testes do Kinect: 61
Testes do MVC: 40
Total geral: 101 testes aprovados.

3.10 Situações não testadas no MVC

Não foram realizados testes unitários com:

- Conexão real do SignalR;
- Firebase real;
- Envio real de e-mail;
- Vários navegadores conectados simultaneamente;
- Queda real da conexão;
- Servidor publicado;
- Aparência visual das páginas;
- Desempenho com grande quantidade de clientes.

Esses comportamentos dependem de recursos externos e devem ser verificados por testes de integração, testes de sistema ou testes manuais.

4. Resultado Geral

Ao todo, foram executados 101 testes unitários com sucesso: 61 testes da aplicação Kinect e 40 testes do sistema MVC, sem testes reprovados nas execuções finais.

5. Conclusão Geral

Os testes do MVC permitiram verificar o processamento das medições recebidas pelo Kinect, a geração de alertas e o envio de atualizações em tempo real.

Também foram verificadas as validações e os dados apresentados pelas ViewModels do sistema.

A suíte identificou uma inconsistência real na quantidade de caracteres permitidos para o token, demonstrando que os testes foram capazes de encontrar um problema que poderia afetar a validação do acesso.



O resultado final de 40 testes aprovados demonstra que os comportamentos avaliados estão funcionando conforme o esperado.

Somados aos 61 testes do aplicativo Kinect, foram executados 101 testes unitários com sucesso, contribuindo para a confiabilidade e manutenção do projeto Inventory Masters.